
Web Application Penetration Test Report — v1.0

Target System: Water is Leben Stiftung Website

Performed by the OpenHack Security Agent

Issued by John Doe (john.doe@example.com)

2026-02-20

Contents

1 Document Control	2
2 Executive Summary	3
3 Execution of the Security Penetration Test	4
3.1 Subject Under Test	4
3.2 Scope	4
3.3 Methodology	4
3.4 Events	5
4 Risk Assessment	6
4.1 CVSS	6
4.2 Risk Categories	7
4.3 Vulnerability States	8
5 Summary of Findings	9
6 Findings and Remediation	10
6.1 Finding	10
6.1.1 SQL Injection Assessment - Keine Schwachstellen gefunden	10
6.2 Finding	11
6.2.1 Fehlende und schwache HTTP Security Header	11
6.3 Finding	13
6.3.1 Account Enumeration über Fehlermeldungen	13
6.4 Finding	14
6.4.1 Information Disclosure: Technologie-Stack und Infrastruktur	14
7 Appendix A - Lists, Scripts and Raw Data	17
8 Appendix B - List of Abbreviations	18

Scope

OpenHack Security Agent was engaged by Water is Leben Stiftung to conduct Nicht-destruktive Web Application Security Assessment IT security investigation. The subject of the investigation was the “Water is Leben Stiftung Website”.

The test was performed using staging environment mirroring production at <https://app.acme.com..>

Objective

The objective of the IT security investigation was to identify vulnerabilities and security gaps which, in the event of misuse, would have an impact on the availability, integrity and confidentiality of the defined applications and/or systems and the data processed on them. The result of this project is a report that contains a management summary of all relevant findings and a compilation of recommended measures.

The technical IT security audit is not a substantive audit effort to ensure that all vulnerabilities and security gaps existing at the time of the audit are identified. There is a possibility that established safeguards will effectively prevent identification and exploitation of vulnerabilities and security gaps. The client acknowledges that this is not an indicator of deficiencies in engagement performance and that additional unidentified vulnerabilities and security gaps may exist.

Disclaimer

‘OpenHack Security Agent’ conducted the security penetration test on behalf of ‘Water is Leben Stiftung’. This report has been prepared exclusively for ‘Water is Leben Stiftung’. It is intended exclusively for internal use and is accordingly not designed to serve third parties as a basis for their decisions, unless agreed otherwise in writing. Third parties may not derive any rights or otherwise benefit from this engagement unless agreed otherwise in writing. Affiliated companies of the client are also considered “third parties” within the meaning of this engagement.

‘OpenHack Security Agent’ does not assume any liability or legal claims against third parties unless agreed otherwise in writing or a disclaimer cannot effectively be implemented.

It is the sole responsibility of anyone who becomes aware of the work results summarized in this report to decide to what extent and in what way the information is useful or appropriate and to supplement, check or update it as part of their own review.

No adjustments will be made to the report to reflect events or circumstances that occur after the report is issued, unless required by law.

The recommendations in this report are general and applicable in many different environments but could have unpredictable effects in specific environments. Therefore, any actions derived from the recommendations should be carefully considered and implemented through the regular change process. This should include appropriate testing of the changes on a test environment prior to going live. If the changes are not tested in advance in an identically configured test environment, failures or other damage cannot be ruled out.

Please note: Technical descriptions (or parts thereof) in this report may be taken from the OWASP Testing Guide (www.owasp.org).

1 Document Control

Document Status

Title	Security Assessment Report
Document Owner	OpenHack Security Agent
Classification	Strictly Confidential
Project Timeframe	2026-02-20

Version History

Version	Date	State	Author	Comments
0.1	2026-02-20	Draft	John Doe	-
1.0	2026-02-20	Final document	John Doe	-

Contact Persons - Assessor

Name	Role	Phone	Email
John Doe	Lead Security Consultant	+1 555 123 4567	john.doe@example.com

Contact Persons - Client

Name	Role	Phone	Email
Jane Smith	Security Lead	+1 555 987 6543	jane.smith@acme.com

2 Executive Summary

OpenHack Security Agent (hereinafter referred to as “ASSESSOR”) has been commissioned by Water is Leben Stiftung (hereinafter referred to as “CLIENT”) to carry out a penetration test.

The penetration test of the Water is Leben Stiftung Website application took place on **2026-02-20**.

For this purpose, the web application, accessible at <https://www.waterislifestiftung.de/>, was tested from the perspective of an external attacker.

Sicherheitsbewertung der Website <https://www.waterislifestiftung.de/> mit Fokus auf nicht-destruktive Testmethoden.

Critical: **0** | High: **0** | Medium: **1** | Low: **2** | Informational: **1** | Total: **4**

Table 2.1: Overview of Findings

Severity Count	
Critical	X
High	X
Medium	X
Low	X
Informational	X
Total	X

A description of the scope and test environment can be found in Chapter 3. Chapter 4 presents the risk assessment methodology. Chapter 5 provides a summary of findings. Chapter 6 contains the detailed technical descriptions of the findings including remediation measures.

It is recommended that the remediation of the critical risk vulnerabilities be treated with the highest priority and countermeasures implemented as soon as possible. The vulnerabilities with high and medium risk should also be remedied in the short term. The low-risk vulnerabilities should be treated with lower priority due to their reduced risk, but should still be addressed.

3 Execution of the Security Penetration Test

3.1 Subject Under Test

3.2 Scope

The scope for the assessment was the following targets:

- <https://www.waterislifestiftung.de/>

3.3 Methodology

The security penetration test of the Water is Leben Stiftung Website took place on 2026-02-20.

OWASP Testing Guide, OWASP Top 10 2021

The web application was tested for security vulnerabilities across the following categories. This can be considered a guideline as penetration tests are a creative process and therefore the tests are not limited to what is given here. The base of these categories is the OWASP Top 10 for Web, which is fully covered by this penetration test approach.

Category	Description
Broken Access Control	Users can act outside their permissions, leading to unauthorized viewing, modification, or deletion of data.
Security Misconfiguration	Systems or cloud services are insecurely configured, e.g., through default passwords or unnecessarily enabled features.
Software Supply Chain Failures	Vulnerabilities arise from compromised third-party code, libraries, or build pipelines.
Cryptographic Failures	Sensitive data is exposed through missing, weak, or incorrectly implemented encryption.
Injection	Attackers inject malicious commands (e.g., SQL or shell code) through input fields that the system erroneously executes.

Category	Description
Insecure Design	Fundamental architectural flaws that exist before implementation make an application inherently insecure.
Authentication Failures	Flaws in identity verification allow attackers to take over sessions or guess passwords (brute force).
Software or Data Integrity Failures	Lack of verification of code or data sources leads to acceptance of untrusted updates or serialization data.
Security Logging & Alerting Failures	Attacks go undetected or responses are delayed due to missing logs or absent alert notifications.
Mishandling of Exceptional Conditions	Programs respond inadequately to unforeseen situations, which can lead to crashes or logic errors.

3.4 Events

During the test, the following restrictions or events occurred that may have affected the scope or depth of the assessment:

4 Risk Assessment

4.1 CVSS

The risk assessment of the vulnerabilities found is performed using the industry standard Common Vulnerability Scoring System v4.0 (hereinafter referred to as “CVSS”). This is a standard that can be used to uniformly assess the vulnerability of computer systems and the severity of security vulnerabilities. This makes it possible to compare vulnerabilities more effectively.

The Common Vulnerability Scoring System has a metric structure and is based on values that are divided into four groups: “Base”, “Threat”, “Environmental” and “Supplemental”. To determine the vulnerability scores, each group has its own rules. The values of the Base group are invariant in time and remain the same in different environments. In the Threat group, the time dependency of a vulnerability is taken into account. For example, the vulnerability of a system to a particular vulnerability decreases over time as more and more countermeasures such as patches become known and available. The Environmental group incorporates various criteria of the specific IT environments into the vulnerability rating. The Supplemental group provides additional context that does not modify the final score.

The CVSS ratings are numerical values on a scale of 0.0 to 10.0, with the highest vulnerability of a system occurring at a value of 10.0. Our rating is based on the CVSS-B (Base Score) and is composed of:

- Attack Vector (AV)
- Attack Complexity (AC)
- Attack Requirements (AT)
- Privileges Required (PR)
- User Interaction (UI)
- Vulnerable System Impact: Confidentiality (VC), Integrity (VI), Availability (VA)
- Subsequent System Impact: Confidentiality (SC), Integrity (SI), Availability (SA)

As penetration testers, we can only assess the general threats posed by a vulnerability (CVSS-B). However, we have no insight into the internal structures of our clients. Therefore, the assessment of the specific risks for the client’s systems and business processes must be done by the client. For this purpose, CVSS offers the Environmental metrics (CVSS-BE). This makes it possible to subsequently adjust the CVSS score of vulnerabilities after the test result has been submitted before they are remedied. This changes the assessment of the risk and thus

the urgency of remediation of a vulnerability. For more information, see [CVSS v4.0 Specification](#).

4.2 Risk Categories

The risk categories used in this report reflect the recommended priority for addressing the vulnerabilities identified during the penetration test. The categorization is based on the probability of exploitation and the significance of the impact. The risk value is calculated using the CVSS v4.0 standard:

Assessment	Risk Category Description
Critical	Critical vulnerabilities that allow an attacker to gain full access to the object under test and its sensitive information. It is possible to cause enormous damage to the client's reputation. Furthermore, these vulnerabilities may allow attackers to gain wide-ranging privileges, including to other systems or applications of the client that have not been investigated in detail within the scope of this project. Exploitation of the vulnerabilities is not prevented and can be reproduced with medium to low effort.
High	Vulnerabilities that may allow an attacker to manipulate sensitive data, damage the client's reputation, gain unauthorized access to sensitive information, or gain unauthorized access to applications or the client's infrastructure. The vulnerabilities are reproducible with medium to high effort.
Medium	Vulnerabilities that may allow an attacker to harm the client's reputation or gain unauthorized access to client data. The privileges that an attacker can gain by exploiting the vulnerabilities are limited.
Low	Security vulnerabilities that do not pose a threat in themselves, but which, for example, provide attackers with useful information about the network and systems. These vulnerabilities allow an attacker only very limited access.
Informational	Anomalies such as functional limitations or inconsistencies. These do not currently represent a risk and have no negative impact on the test object. Accordingly, no action is required. Nevertheless, countermeasures can be helpful for the functional and safety improvement of the test object. If necessary, this may give rise to risks under other circumstances.

4.3 Vulnerability States

The vulnerabilities can be in one of the following states:

- **Active:** The vulnerability has been identified and it has been possible to verify its existence through exploitation or direct evidence.
- **Potential:** The vulnerability has been identified but its exploitation has not been possible, so its existence cannot be fully verified, and it is up to the client to determine the impact.
- **Fixed:** The vulnerability has been remediated and verified through retesting.

DRAFT

5 Summary of Findings

Id	Finding Name	Risk Assessment	CVSS v4 Score
01	SQL Injection Assessment - Keine Schwachstellen gefunden	Informational	0
02	Fehlende und schwache HTTP Security Header	Medium	5.3
03	Account Enumeration über Fehlermeldungen	Low	4.3
04	Information Disclosure: Technologie-Stack und Infrastruktur	Low	2.3

6 Findings and Remediation

6.1 Finding

6.1.1 SQL Injection Assessment - Keine Schwachstellen gefunden

Severity	Informational
CVSS Base Score	0.0
Assets	N/A
Status	closed

Description

Umfassende SQL Injection Tests wurden auf <https://www.waterislifestiftung.de/> durchgeführt. Keine SQL Injection Schwachstellen wurden entdeckt. Die Website verwendet das IONOS MyWebsite CMS mit korrekter Eingabevalidierung und implementiert WAF-Schutz.

Durchgeführte Tests: - Error-Based SQL Injection - Boolean-Based Blind SQL Injection - Time-Based SQL Injection (SLEEP, BENCHMARK, pg_sleep) - Union-Based SQL Injection - Header-basierte SQL Injection

Testergebnisse: - Keine SQL-spezifischen Fehlermeldungen - Identische Antwortzeiten für wahre/falsche Bedingungen (~0.13-0.23s) - Keine Verzögerungen bei Time-Based Payloads - Vermutlich parametrisierte Queries oder ORM-Nutzung

WAF-Erkennung: - WAF ist aktiv und blockiert SQL-Payloads - Response-Code: 503 bei verdächtigen Anfragen - Ausgelöst durch: X-Forwarded-For Header mit SQL-Inhalten

Proof of Concept

Getestete Endpoints: - /kontakt-impressum/ (Formular-Parameter) - / (Homepage mit URL-Parametern)

Getestete Payloads:

```
' (single quote)
' OR '1'='1
' AND 1=1 --
```

```
' AND 1=2 --
1' AND SLEEP(1) --
1' AND BENCHMARK(1000000,MD5(1)) --
1' UNION SELECT 1 --
```

WAF-Blockade: - X-Forwarded-For: 127.0.0.1' → 503 Service Unavailable - X-Forwarded-For: 127.0.0.1' OR '1'='1 → 503 Service Unavailable

Remediation

Keine unmittelbaren Maßnahmen erforderlich.

Die Website ist gegen SQL Injection Angriffe geschützt durch: 1. ☑ WAF (Web Application Firewall) aktiv 2. ☑ Korrekte Eingabevalidierung 3. ☑ Vermutlich parametrisierte Queries oder ORM 4. ☑ CAPTCHA-Schutz im Kontaktformular

Empfohlene laufende Maßnahmen: - Regelmäßige Sicherheitsupdates für das CMS - Überwachung der WAF-Regeln - Content Security Policy (CSP) Header implementieren - Logging und Monitoring für verdächtige Aktivitäten

6.2 Finding

6.2.1 Fehlende und schwache HTTP Security Header

Severity	Medium
CVSS Base Score	5.3 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:L/VA:N/SC:N/SI:N/SA:N)
Assets	N/A
Status	open

Description

Die Website <https://www.waterislifestiftung.de/> weist mehrere kritische Defizite bei den HTTP Security Headern auf. Essenzielle Schutzmechanismen wie HSTS (Strict-Transport-Security), X-Frame-Options und Referrer-Policy fehlen komplett. Die vorhandene Content-Security-Policy (CSP) befindet sich im Report-Only-Modus und erlaubt unsichere Direktiven wie 'unsafe-inline', 'unsafe-eval' und Wildcard-Quellen (*).

Identifizierte Probleme: 1. **Fehlendes HSTS:** Ermöglicht SSL-Stripping und Downgrade-Angriffe 2. **Fehlende X-Frame-Options:** Ermöglicht Clickjacking-Angriffe 3. **CSP im Report-Only-Modus:** Bietet keinen aktiven Schutz, nur Monitoring 4. **Unsichere CSP-Direktiven:** 'unsafe-inline' und 'unsafe-eval' erlauben XSS-Execution 5. **Cookie ohne**

HttpOnly: Das DIY_SB-Cookie ist via JavaScript zugänglich (XSS-Risiko) 6. **SameSite=None:** CSRF-Risiko durch fehlenden Cookie-Schutz

Cookie-Attribute: - DIY_SB: path=/; SameSite=None; secure (kein HttpOnly)

Verfügbare Header: - ☒ X-Content-Type-Options: nosniff (korrekt) - ☒ X-XSS-Protection: 0 (deprecated) - ☒ Strict-Transport-Security: fehlend - ☒ X-Frame-Options: fehlend - ☒ Referrer-Policy: fehlend - ☒ CSP: Report-Only mit unsicheren Direktiven

Proof of Concept

HTTP Response Header (Auszug):

```
X-Content-Type-Options: nosniff
X-XSS-Protection: 0;report=https://cdn.website-start.de/app/reporting/policyvi
Content-Security-Policy-Report-Only: default-src *;script-
src 'unsafe-inline' 'unsafe-eval' *;style-src 'unsafe-inline' *;connect-
src * blob;;
```

CSP Problem: - script-src 'unsafe-inline' 'unsafe-eval' * erlaubt beliebige Inline-Scripts und Eval - default-src * erlaubt Ressourcen von beliebigen Domains - Nur Report-Only, nicht enforcing

Cookie-Analyse:

```
DIY_SB=dd5163bcdac024226b3e582fff787d28; path=/; SameSite=None; secure
```

- ☒ Kein HttpOnly-Flag → JavaScript kann Cookie auslesen
- ☒ SameSite=None → Kein CSRF-Schutz
- ☒ Secure-Flag → Nur über HTTPS

Remediation

Priorität: HOCH

1. HSTS implementieren:

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
```

2. CSP in enforcing-Modus mit restriktiven Direktiven:

```
Content-Security-Policy: default-src 'self';
script-src 'self';
style-src 'self' 'unsafe-inline';
img-src 'self' data: https;;
font-src 'self';
connect-src 'self';
frame-ancestors 'none';
```

```
base-uri 'self';  
form-action 'self';
```

3. X-Frame-Options hinzufügen:

```
X-Frame-Options: DENY
```

4. Cookie-Sicherheitsattribute verbessern:

```
DIY_SB=<value>; path=/; SameSite=Lax; Secure; HttpOnly
```

5. Referrer-Policy hinzufügen:

```
Referrer-Policy: strict-origin-when-cross-origin
```

6.3 Finding

6.3.1 Account Enumeration über Fehlermeldungen

Severity	Low
CVSS Base Score	4.3 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N)
Assets	N/A
Status	open

Description

Das IONOS-Login-System gibt durch spezifische Fehlermeldungen preis, ob ein Benutzerkonto existiert oder nicht. Bei Eingabe einer nicht existierenden E-Mail-Adresse wird die Meldung “Kein IONOS Konto mit dieser E-Mail-Adresse gefunden” angezeigt. Bei einer gültigen Domain oder E-Mail erfolgt hingegen eine Weiterleitung zur Zwei-Faktor-Authentifizierungsseite.

Diese Diskrepanz ermöglicht es Angreifern, gültige von ungültigen Accounts zu unterscheiden und so gezielt vorhandene Accounts für Brute-Force- oder Credential-Stuffing-Angriffe zu identifizieren.

Proof of Concept

Getestete Szenarien:

1. Nicht existierende E-Mail:

- Eingabe: nichtexistierend12345@beispiel.de

- Antwort: “Kein IONOS Konto mit dieser E-Mail-Adresse gefunden. Korrigieren Sie Ihre Eingabe oder melden Sie sich mit Ihrer Kundennummer oder Domain an.”
- Response-Code: 200 OK (bleibt auf Login-Seite)

2. Gültige Domain:

- Eingabe: `www.waterislifestiftung.de`
- Antwort: Weiterleitung zur 2FA-Seite (`/emailconfirmation`)
- Response-Code: 302 Redirect

Unterschiedliche Antwortzeiten: - Ungültiger Account: ~1.2s - Gültiger Account: ~0.8s (Weiterleitung)

Remediation

Empfohlene Maßnahmen:

1. Generische Fehlermeldungen verwenden:

- Zeige immer dieselbe Meldung an: “Falls ein Konto existiert, wurde ein Bestätigungscode gesendet.”
- Keine Unterscheidung zwischen existierenden und nicht-existierenden Accounts

2. Identische Antwortzeiten implementieren:

- Füge künstliche Verzögerungen hinzu, um Timing-Angriffe zu verhindern
- Normalisiere die Server-Response-Zeit

3. Weiterleitung zur 2FA-Seite für alle Anfragen:

- Leite immer zur E-Mail-Bestätigungsseite weiter
- Sende Bestätigungscode nur für existierende Accounts
- Zeige keine Fehlermeldung an, wenn keine E-Mail versendet wurde

6.4 Finding

6.4.1 Information Disclosure: Technologie-Stack und Infrastruktur

Severity	Low
CVSS Base Score	2.3 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N)
Assets	N/A
Status	open

Description

Die Website enthüllt durch Meta-Tags, HTML-Kommentare und Inline-JavaScript Informationen über das verwendete Content-Management-System, die Hosting-Infrastruktur und interne System-URLs. Diese Informationen können von Angreifern genutzt werden, um gezielte Angriffe auf bekannte Schwachstellen des IONOS MyWebsite-Systems zu planen.

Preisgegebene Informationen: - CMS: IONOS MyWebsite - Hosting-Provider: 1&1 IONOS - System-URL: <https://120.sb.mywebsite-editor.com/> - CDN-URL: cdn.website-start.de - Session-Namespace: DIY_SB - Template-ID: 2134 - Render-Zeitstempel: Wed, 18 Feb 2026 06:53:24 +0100 - Admin-Login: <https://login.1and1-editor.com/132362044/>

Proof of Concept

Quelltext-Analyse:

1. Meta-Generator-Tag:

```
<meta name="generator" content="IONOS MyWebsite">
```

2. HTML-Kommentare mit Zeitstempel:

```
<!-- rendered at Wed, 18 Feb 2026 06:53:24 +0100 -->
```

3. Inline-JavaScript mit System-URLs:

```
var diy = {  
  config: {  
    renderUrl: 'https://120.sb.mywebsite-editor.com/',  
    cdnUrl: 'https://cdn.website-start.de/',  
    sessionNamespace: 'DIY_SB'  
  }  
};
```

4. Impressum-Login-Link:

```
<a href="https://login.1and1-editor.com/132362044/www.waterislifestiftung.de/de?pageId=306234">  
  Login  
</a>
```

Remediation

Empfohlene Maßnahmen (Security by Obscurity):

1. Meta-Generator-Tag entfernen oder modifizieren:

```
<!-- Entfernen oder generisch gestalten: -->  
<meta name="generator" content="CMS">
```

2. HTML-Kommentare mit Zeitstempeln entfernen:

- Entferne den Kommentar `<!-- rendered at ... -->`
- Vermeide Debug-Informationen im Produktivsystem

3. Interne System-URLs aus JavaScript extrahieren:

- Lade Konfigurationsdaten serverseitig
- Vermeide Preisgabe interner URLs im Client-Code

4. Login-Link aus dem öffentlichen Impressum entfernen:

- Verlange Authentifizierung für Admin-Links
- Oder entferne den Link komplett

DRAFT

7 Appendix A - Lists, Scripts and Raw Data

DRAFT

8 Appendix B - List of Abbreviations

Abbreviation	Description
API	Application Programming Interface
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart
CVSS	Common Vulnerability Scoring System
FTP	File Transfer Protocol
IDOR	Insecure Direct Object Reference
MD5	Message-Digest Algorithm 5
OAuth	Open Authorization
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
PII	Personally Identifiable Information
SQL	Structured Query Language
TOTP	Time-based One-Time Password
URI	Uniform Resource Identifier
WAF	Web Application Firewall

+-----+-----+